



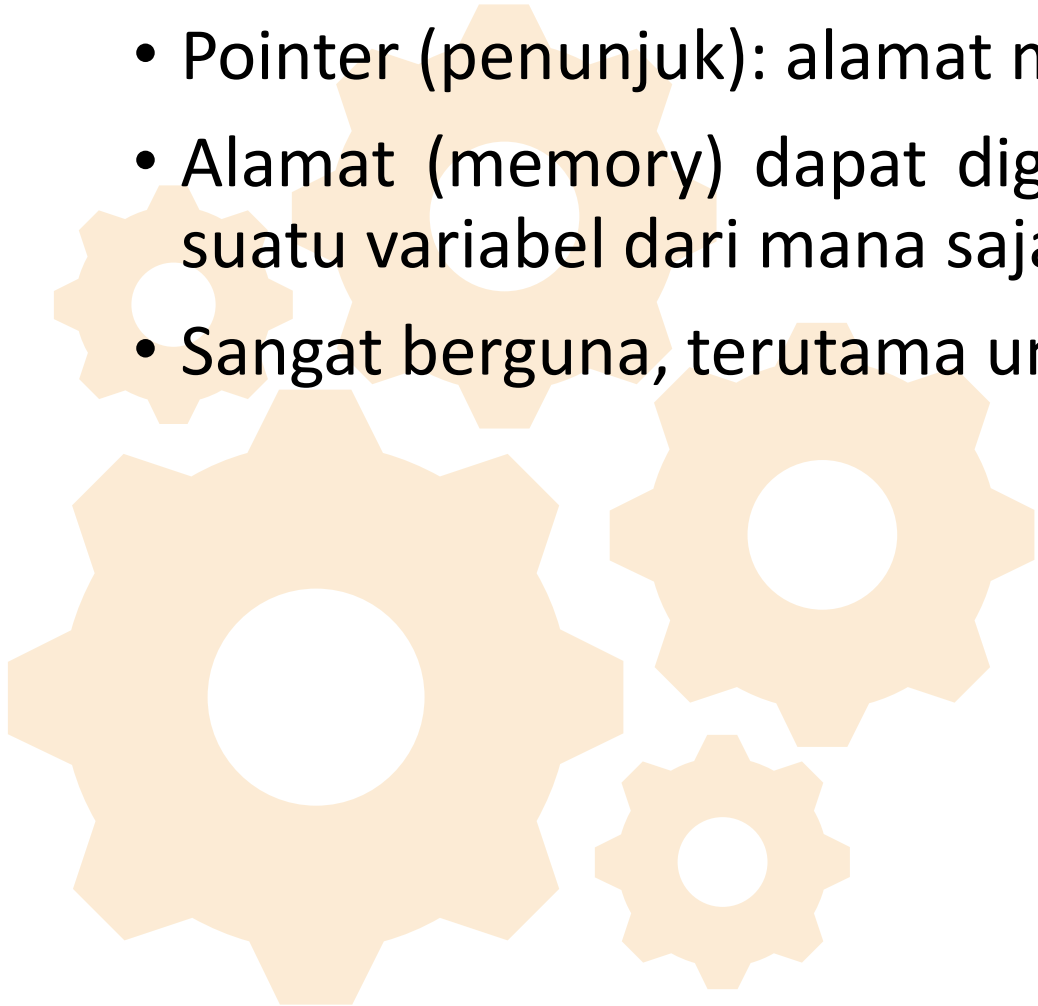
Pointer

Struktur Data



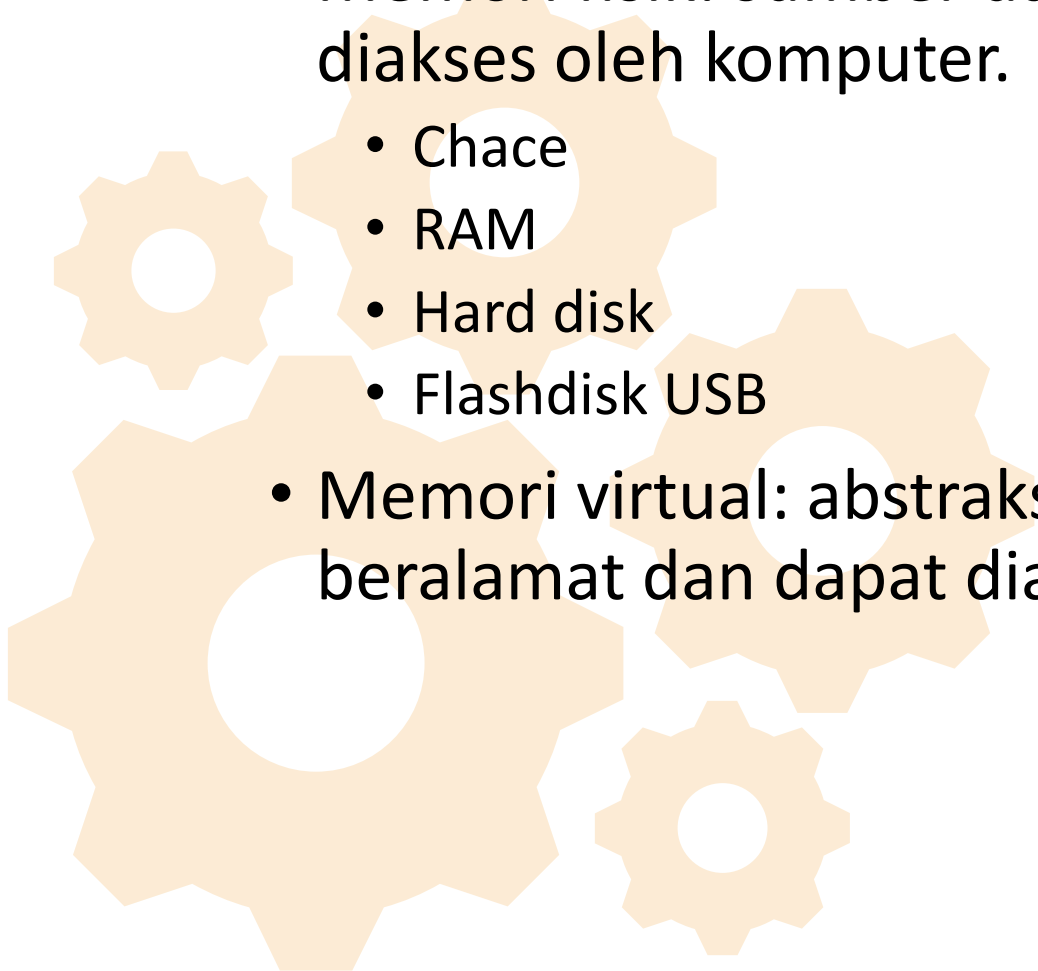
Pointer dan pengalamatan

- Pointer (penunjuk): alamat memory dari variabel
- Alamat (memory) dapat digunakan untuk mengakses atau merubah suatu variabel dari mana saja.
- Sangat berguna, terutama untuk struktur data.



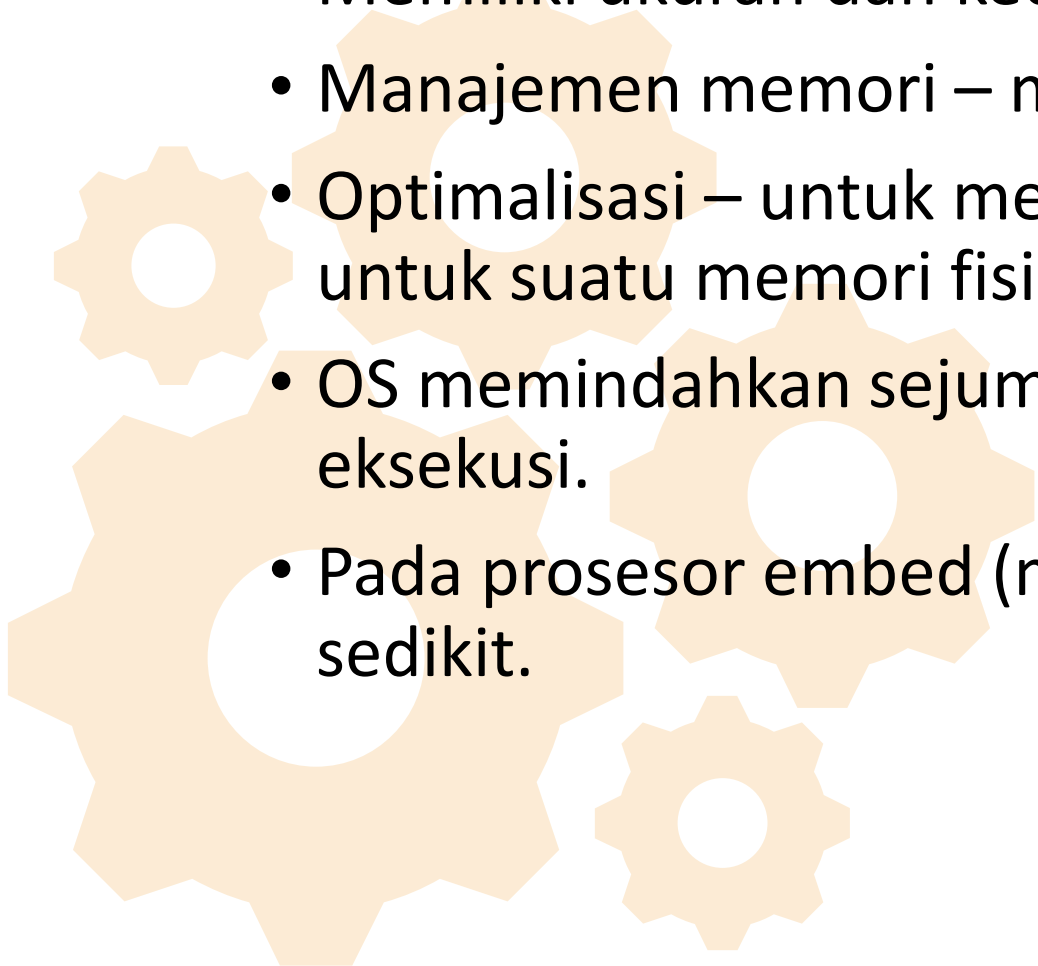
Physical dan virtual memory

- Memori fisik: sumber daya fisik di mana data dapat disimpan dan diakses oleh komputer.
 - Cache
 - RAM
 - Hard disk
 - Flashdisk USB
- Memori virtual: abstraksi dari sistem operasi, memiliki ruang yang beralamat dan dapat diakses dengan suatu kode program.



Apa yang penting dari memori fisik?

- Memiliki ukuran dan kecepatan yang berbeda-beda.
- Manajemen memori – merupakan fungsi utama OS.
- Optimalisasi – untuk menjaga kode kita terbaik yang digunakan untuk suatu memori fisik yang tersedia.
- OS memindahkan sejumlah data pada memori fisik selama eksekusi.
- Pada prosesor embed (mikrokontroler) mungkin memorinya sedikit.



Memori virtual

- Berapa banyak memori fisik yang kita punya?
 - 2 MB cache + 2 GB RAM + 100 GB (HDD)+ ...
- Berapa banyak virtual memori yang kita punya?
 - < 4 GB (untuk OS 32 bit), biasanya 2 GB pada Windows, 3-4 GB pada linux.
- Virtual memori memetakan dirinya pada bagian dari memori fisik.
- Bagian yang berguna bagi memori virtual: *stack* dan *heap*.
- Stack: dimana variabel sudah dideklarasikan (tetap)
- Heap: dimana memori dinamis digunakan

Mengalamatkan variable

- Setiap variabel yang berada pada suatu memori memiliki alamat!
- Apa yang tidak memiliki alamat?
 - Variabel register.
 - Konstan/literal/preprocessor/macros
 - Ekspresi
- Bagaimana cara mencari alamat dari suatu variabel? Dengan operator “&”.

`int n = 4;`

`float pi = 3.14159;`

`int *pn = &n; /* berisi alamat dari integer n */`

`float *ppi = π /* berisi alamat dari float pi */`

- Alamat dari suatu variabel yang bertipe t memiliki tipe t^* .

Memahami pointer

- Saya sudah memiliki pointer.. Terus gmn?
- Mengakses/memodifikasi alamat variabel:
- Memahami operator pointer (*).

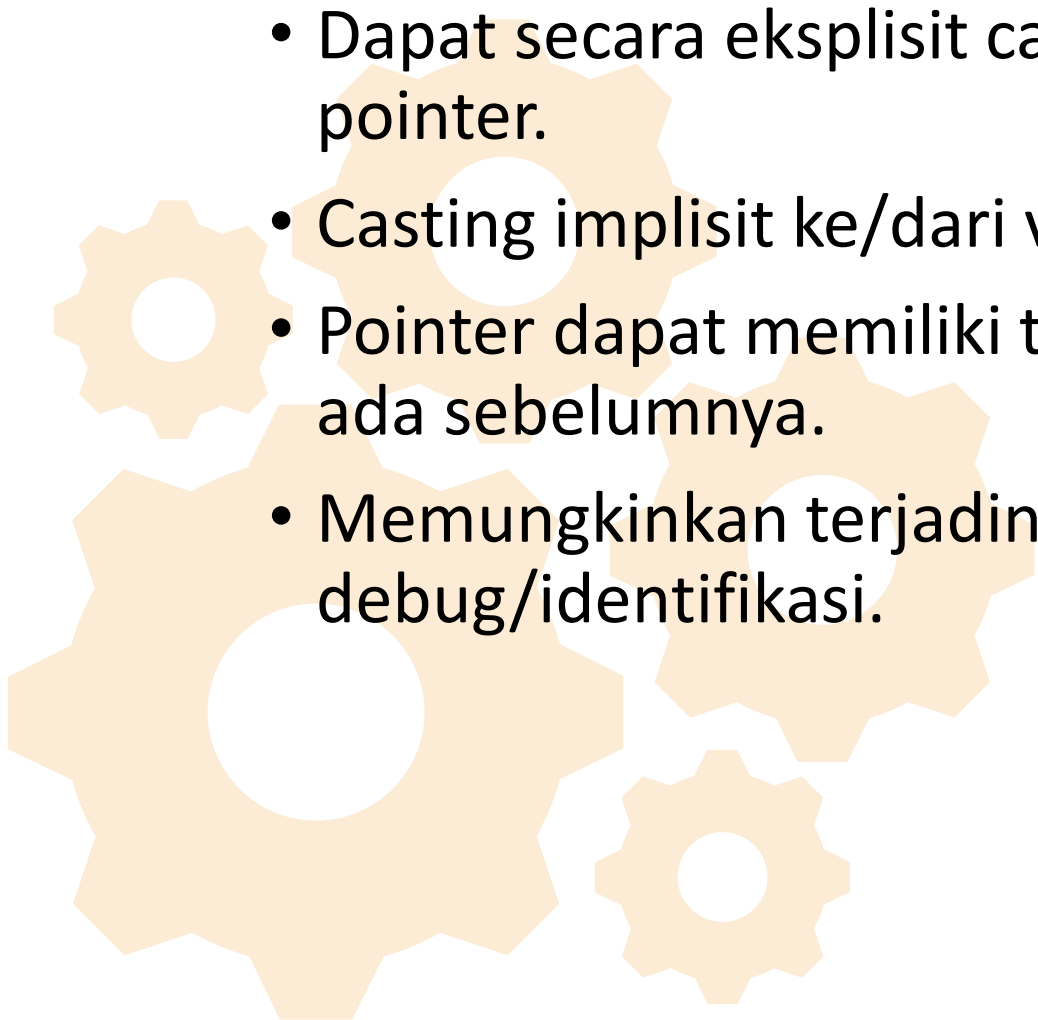
```
printf("pi = %g\n",*ppi); /* outputnya pi = 3.14159 */
```

```
*ppi = *ppi + *pn; /*outputnya pi sekarang 7.14159 */
```

- Memahami pointer seperti memahami variabel lain.
- Null pointer: artinya: pointer tidak memiliki referensi ke siapapun.

Casting pointer

- Dapat secara eksplisit casting ke suatu tipe pointer pada suatu tipe pointer.
- Casting implisit ke/dari void * dapat juga dilakukan.
- Pointer dapat memiliki tipe data baru dari tipe data yang sudah ada sebelumnya.
- Memungkinkan terjadinya tabrakan memori dan susah di debug/identifikasi.

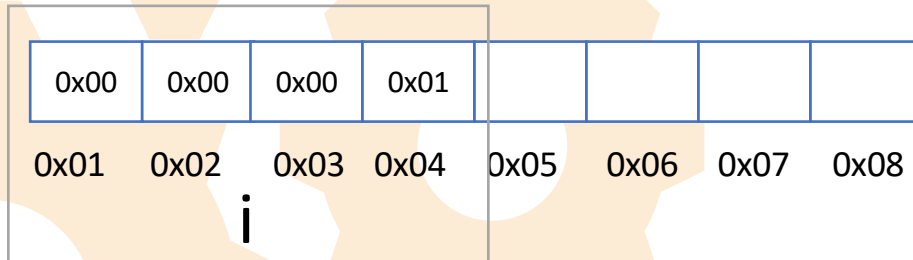


Intuisi

- Setiap byte memiliki nilai dan alamat memori yang ditempati.



- kapan menyimpan variabel di memori, contoh: `int i = 1`.



`i = 1`

`&i = 0x01`

& operator: get address of a variable

Intuisi

- Untuk menyimpan alamat variabel menggunakan tipe spesial yang disebut dengan pointer.

`int *pi;`

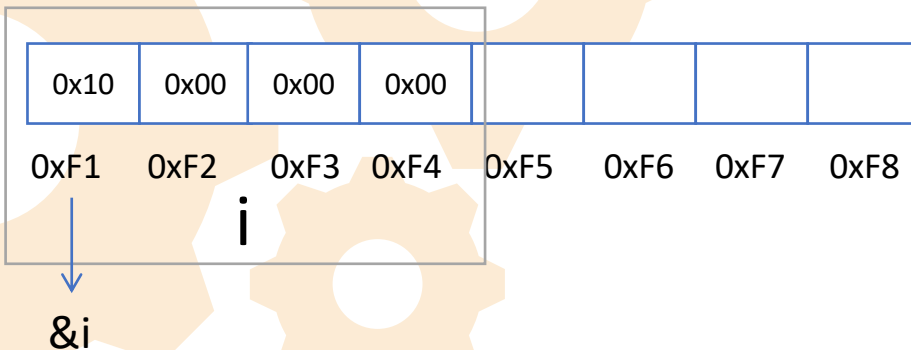
↑
Pointer dari
variabel integer

`char *pc;`

↑
Pointer dari
variabel char

`float *pf;`

↑
Pointer dari
variabel float



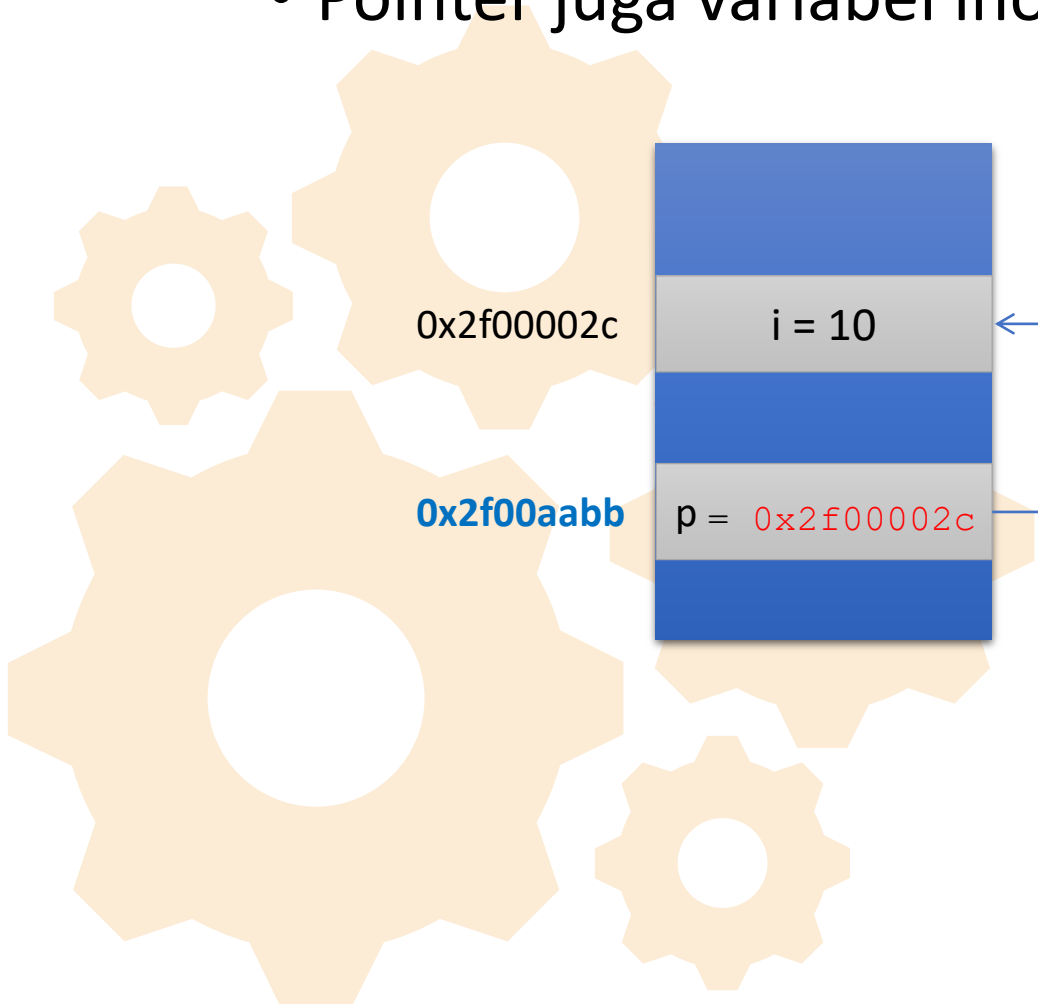
```
int *pi = &i;
```



```
int* pi;  
pi = &i;
```

Intuisi

- Pointer juga variabel lho, jadi dia disimpan didalam memori juga.



```
int i = 10;  
int *p = &i;
```

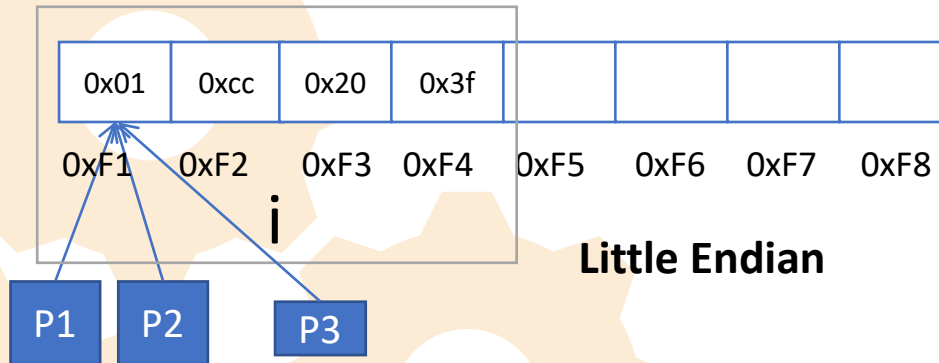
```
p    = 0x2f00002c  
&p   = 0x2f00aabb  
*p   = 10
```

?

`*p` : mendapat nilai dari alamat yang ditunjuk p

Intuisi

- Tipe pointer memberitahukan cara bagaimana mendapatkan nilai yang ditunjuk oleh pointer tersebut.



```
int i = 0x3f20cc01;  
char *p1 = (char *)&i;  
short *p2 = (short *)&i;  
int *p3 = &i;
```

- p1 menunjuk ke char-block. $*p1 = 0x01$
- p2 menunjuk ke short-block. $*p2 = 0xCC01$
- p3 menunjuk ke int-block. $*p3 = 0x3f20cc01$

Endian

- **Big-endian** dan **little-endian** mengacu pada byte-byte memori mana yang akan lebih signifikan pada tipe data multi-byte.
- Sebagai contoh, menyimpan nilai 1025 pada suatu memory (4 bytes).

Big endian

0000.0000

0000.0000

0000.0100

0000.0001

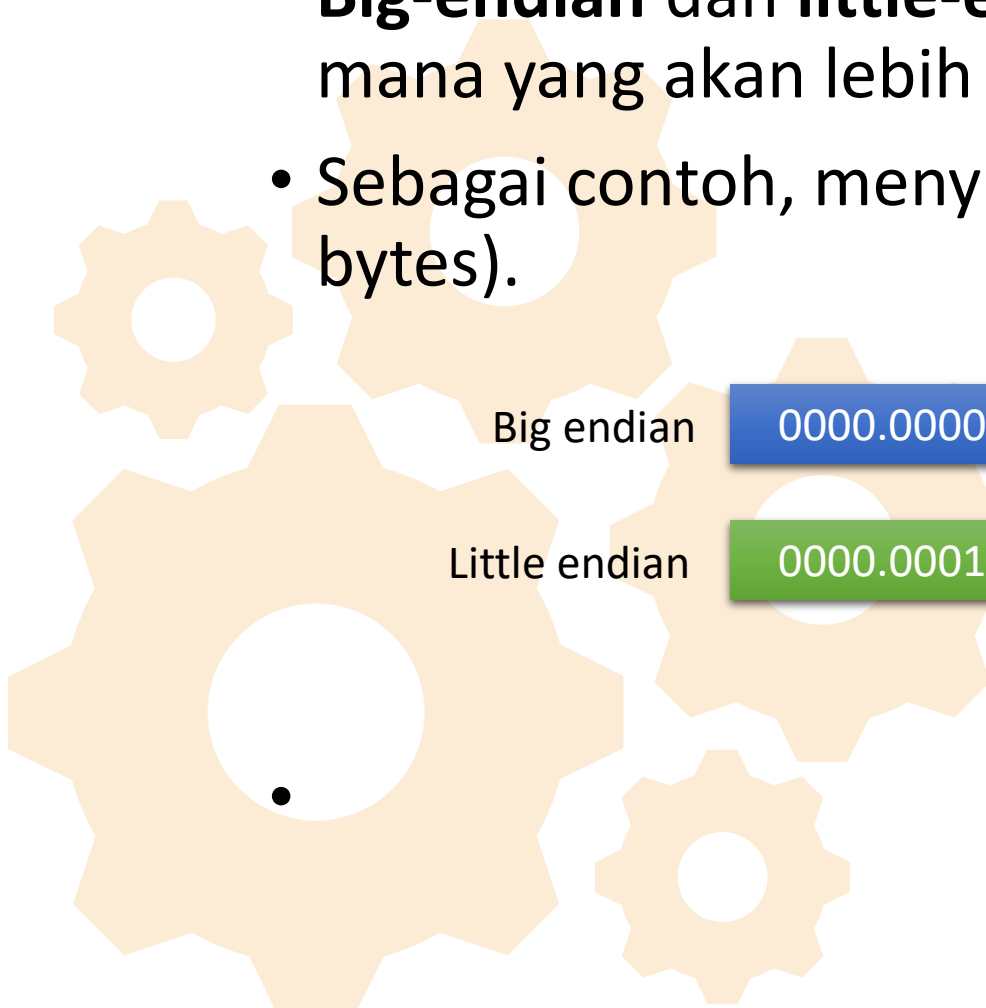
Little endian

0000.0001

0000.0100

0000.0000

0000.0000



Pointer (lanj.)

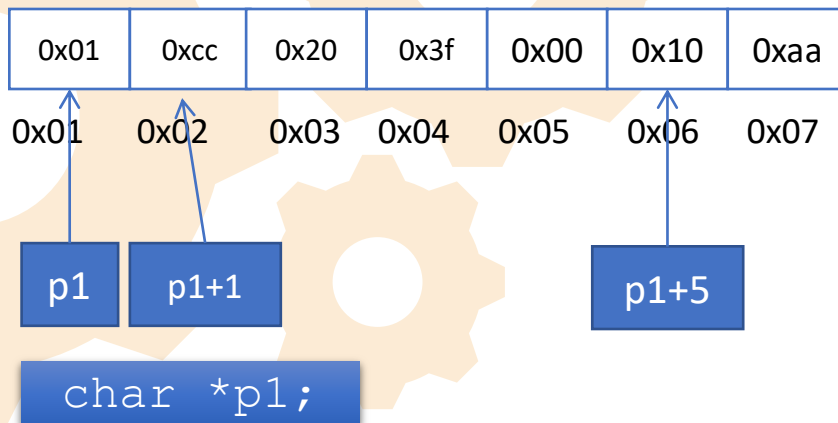
operatorn sizeof

- ❑ Ukuran dari pointer tidak bergantung dari tipe dari pointer.
- ❑ Ukuran pointer bergantung dari processor (16 bits, 32 bits, 64 bits)
 - Untuk windows 32 bits: ukuran dari pointer adalah 4 bytes

```
int main()  
{  
    char c = 0;  
    char *p = &c;  
    printf("size = %d", sizeof(p));  
}
```

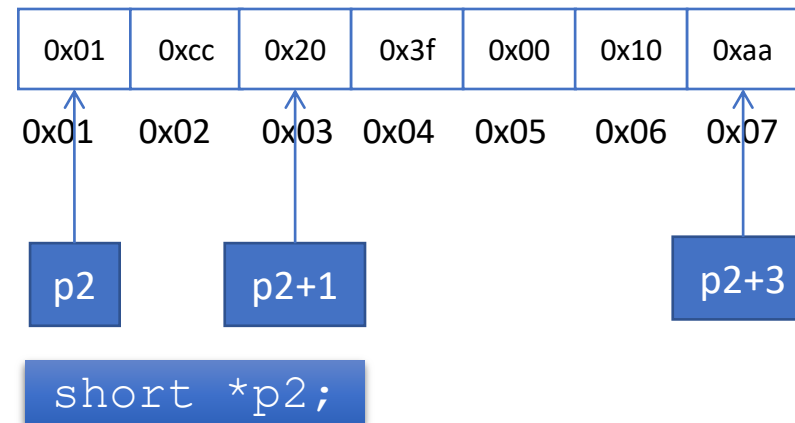
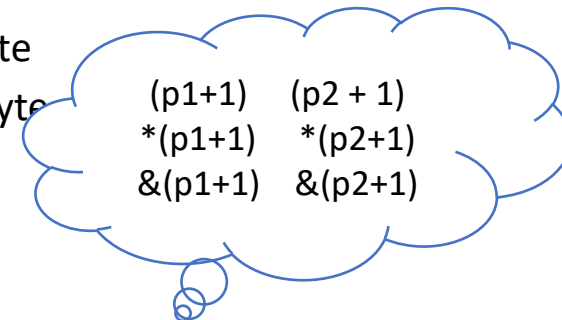
Operator Pointer

Operator	deskripsi	Contoh
+	move forward n steps	<code>p += 10;</code>
-	move backward n step	<code>p -= 1;</code>
++	move forward 1 step	<code>p++;</code>
--	move backward 1 step	<code>p--;</code>



❑ Catatan: setiap langkah adalah setiap jarak k bytes yang bergantung dari tipe pointer.

- byte: 1 byte
- Short: 2 byte
-



Pointer pointer operator - Latihan

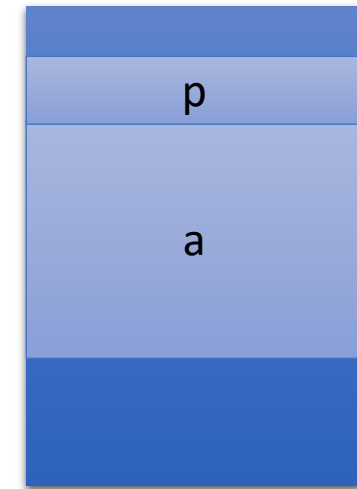
```
char a[6] = {10, 20, 30, 40, 50, 60};  
char *p = a;
```

0x001cff04
0x001cff08

a = ?
&a = ?
*a = ?

p = ?
&p = ?
*p = ?

p + 1 = ?
(*p) + 1 = ?
*(p + 1) = ?
&p + 1;
&a + 1
a++; a = ?
p++; p = ?



Pointer ke pointer

- ❑ Perhatikan bahwa, setiap variabel pointer adalah sebuah **variable**.
- ❑ Untuk menyimpan alamat dari sebuah variabel pointer , kita menggunakan variabel pointer-ke-pointer.

```
int iVar = 10;  
int *p1 = &iVar;  
int **p2 = &p1;
```

*p1 == ?
*p2 == ?
*(*p2) == ?

0x100	iVar = 10
0x200	p1 = 0x100
0x300	p2 = 0x200

Pointer- Alokasi Dinamis

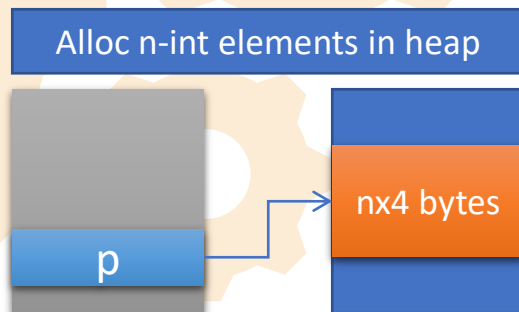
❑ Alokasi Statis:

```
int a = 10;  
int array[1000];
```

- Variabel akan di alokasikan di stack → dengan ukuran terbatas
- Banyaknya elemen dari array adalah konstanta
- Dapat tidak dibersihkan memorynya ketika mereka menjadi tidak berguna

❑ Alokasi Dinamis

- Pointer user
- Alokasi sebuah blok memory di heap → kapasitas besar
- Dihapus dengan mudah



Pointer

Alokasi Dinamis (cont.)

- ❑ Ada dua cara untuk alokasi dinamis.

Old C style

- Using `stdlib.h`
- Using `malloc/free`

```
int main()
{
    char *i = (char*) malloc (100);

    // some code here

    free(i);
}
```

C++ style

- Using `new/delete`
- Using `new[] / delete[]`

```
int main()
{
    char *i = new char[100];

    // some code here

    delete []i;
}
```

Pointer

Alokasi Dinamis (cont.)

- ❑ Gunakan delete untuk new,
- ❑ Gunakan delete[] untuk new[]

```
struct A
{
public:
    static int count;
    int val;
    A()
    {
        printf("Created %d\n",
            val = count++);
    }
    ~A()
    {
        printf("Deleted %d\n",
            val);
    }
};

int A::count = 0;
```

```
int main()
{
    A *cA = new A[10];
    delete cA;
    return 1;
}
```

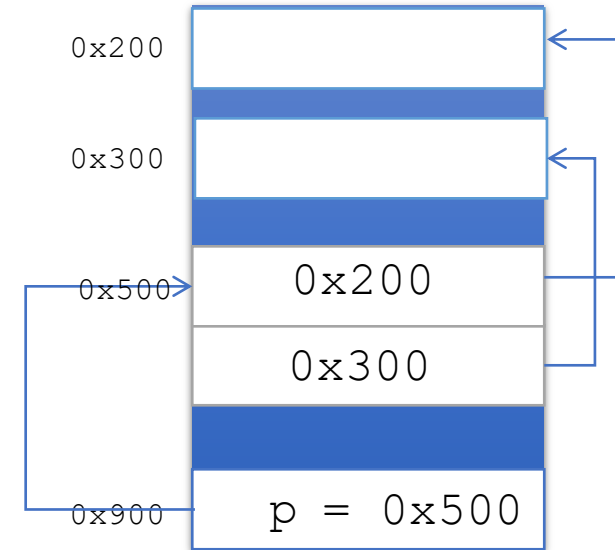
Delete cA[0] only

```
int main()
{
    A *cA = new A[10];
    delete []cA;
    return 1;
}
```

Delete all cA

Pointer-to-pointer Alokasi Dinamis

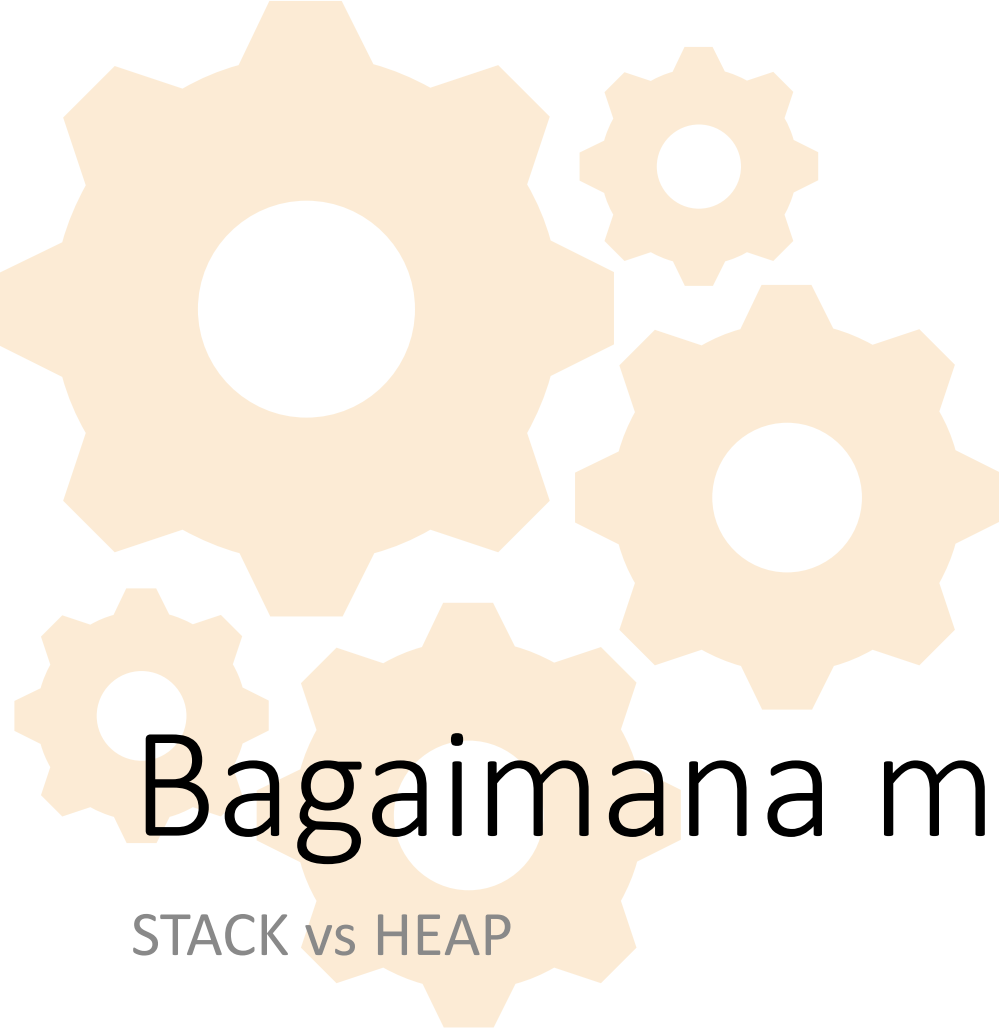
```
int **p;  
p = new int*[2];  
*(p+0) = new int;  
*(p+1) = new int;
```



□ Secara umum, digunakan untuk alokasi array 2D

```
int **p = new int*[3];  
p[0] = new int[4];  
p[1] = new int[4];  
p[2] = new int[4];
```

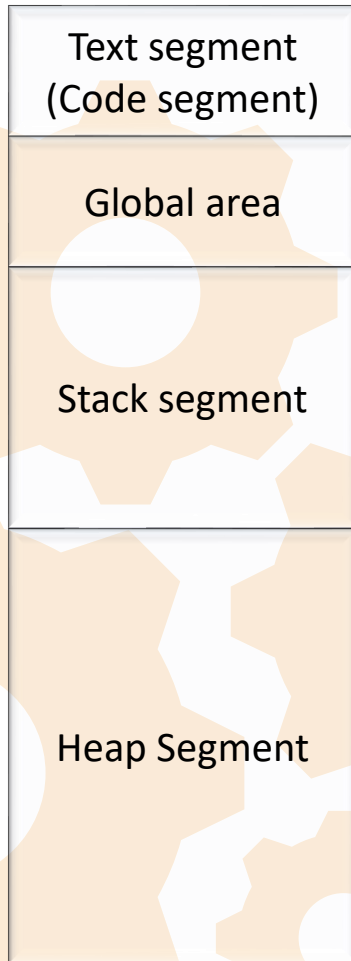
$*(*(p + i) + j) \Leftrightarrow p[i][j]$



Bagaimana memory terstruktur?

STACK vs HEAP

Penyimpanan Run-time



❑ Code segment:

- Dimana program terkompilasi berada di memory

❑ Global area:

- Menyimpan variabel global

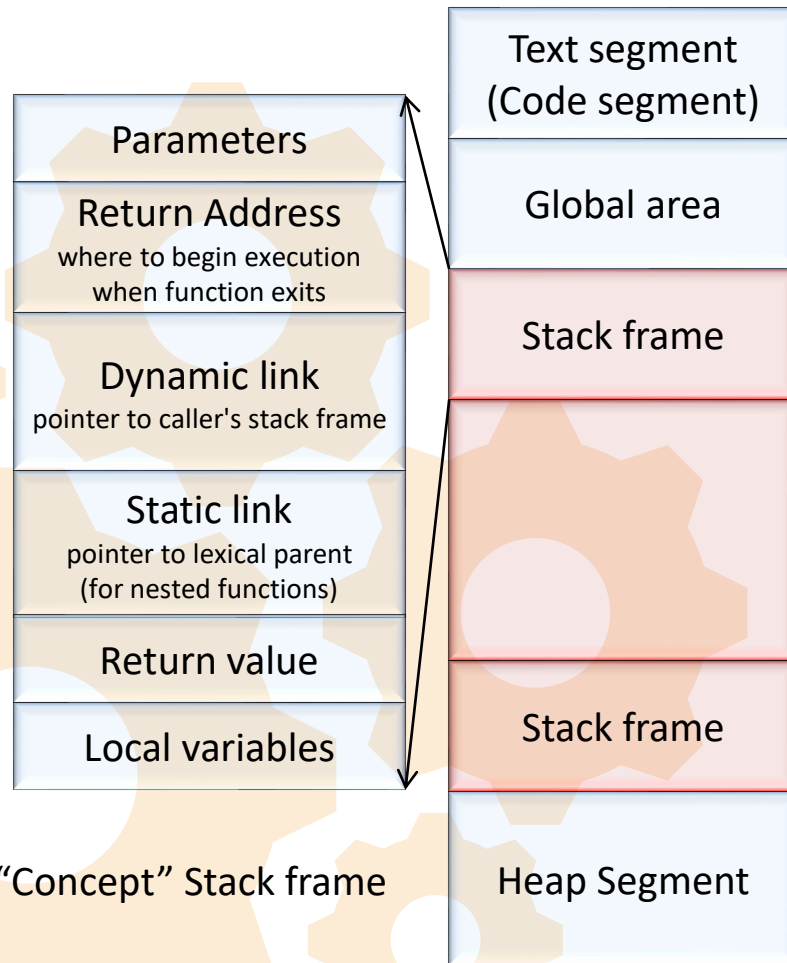
❑ Stack segment:

- Dimana parameter-parameter dan variabel variabel lokal di alokasikan

❑ Heap segment:

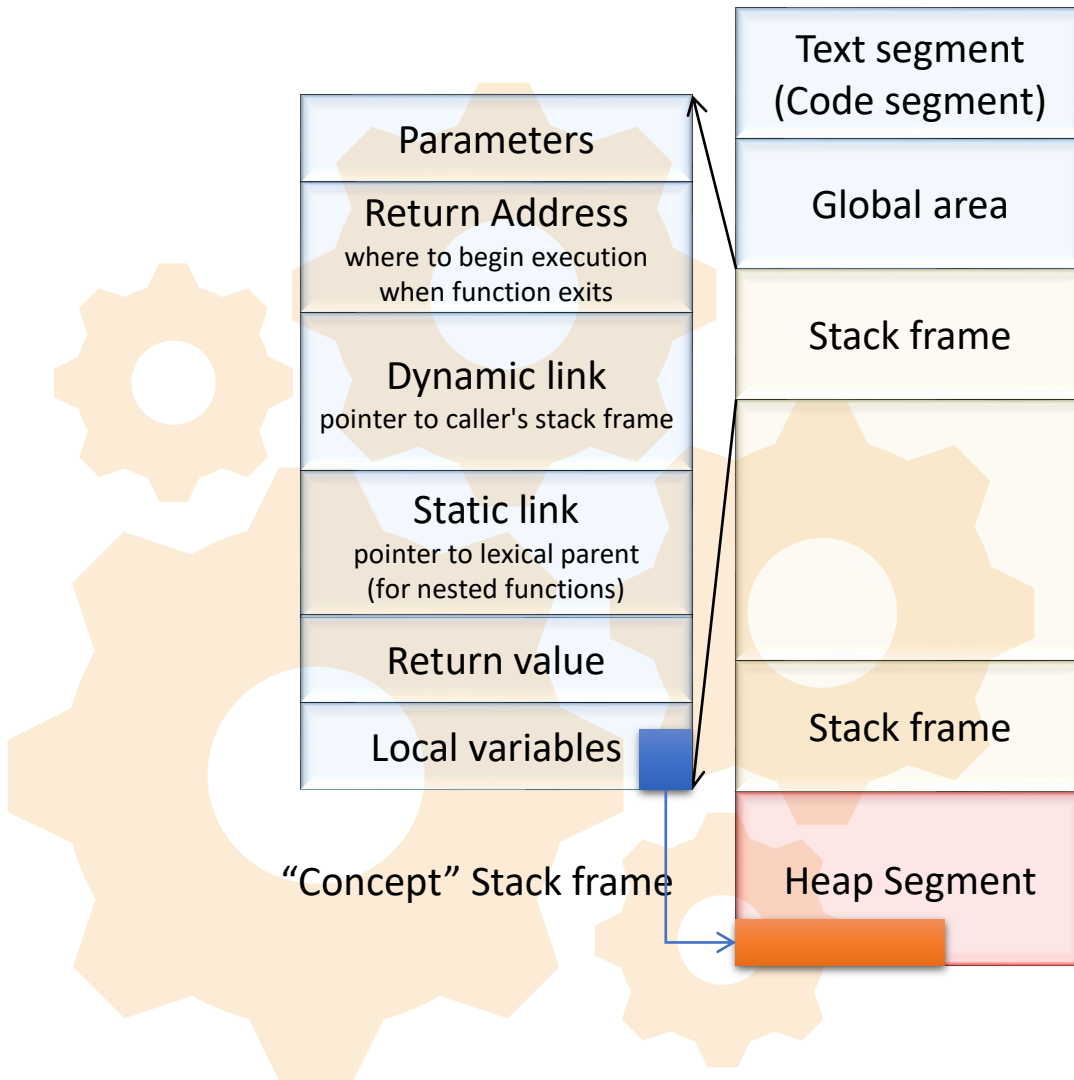
- Dimana variabel variabel yang teralokasi secara dinamis dialokasikan

Stack



- ❑ Dimana Parameter dan variabel lokal dialokasikan
- ❑ Ukuran terbatas → **Stack overflow**
- ❑ Memory yang digunakan di stack adalah sementara dan akan otomatis dilepaskan
- ❑ Pemrosesan cepat/ukuran sedikit

Heap



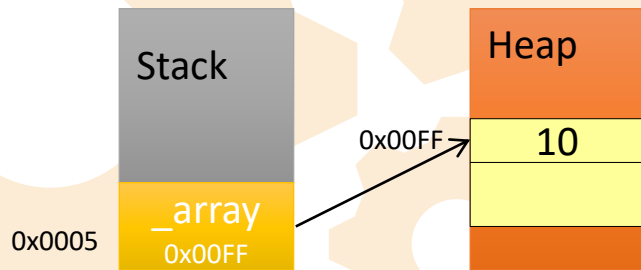
- ❑ Memory pool yang besar
- ❑ Alokasi Dinamis
- ❑ Tetap teralokasi sampai secara spesifik di dealokasikan → **bisa leak !**
- ❑ Harus diakses melalui pointer
- ❑ Array yang besar, struktur data, atau class seharusnya disimpan di Heap
- ❑ Besar dan dinamis

Heap vs Stack

`int _array[10];` → disimpan di stack

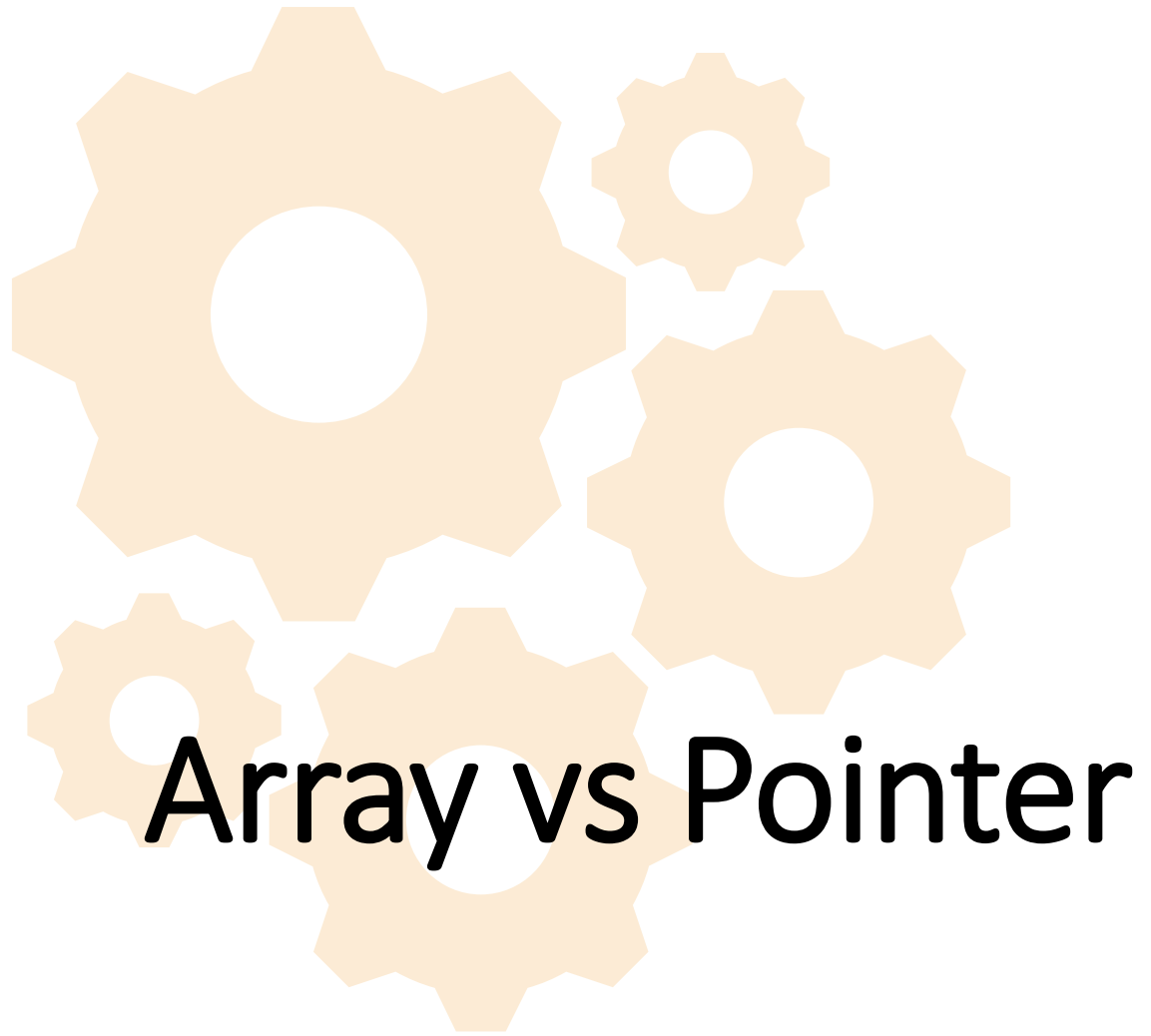
`int *_array = new int[n]`

- Pointer `_array` disimpan di Stack
- Data dari array disimpan di heap



Value of:

- `_array` : address where int "point" into in heap (0x00FF)
- `(*_array)`: value at it's address on heap (10)
- `(&_array)`: address of the memory which used for stored pointer `_array` in stack (0x0005)



Array vs Pointer

Array dan Pointer

- **Array** adalah koleksi objek dengan type yang sama yang diacu menggunakan *single name*
- **Pointer** adalah variable dengan nilai dari alamat memory yang dapat menjadi acuan ke variable lainnya berdasarkan type yang diberikan
- Array adalah pengelompokan dari ***contiguous memory***
- array itu sendiri sebagai pointer ke lokasi memory pertama dalam ***contiguous memory***

Array dan Pointer

- Meskipun terlihat berbeda, array dan pointer memiliki keceratan yang terkait.
- Passing array menetapkan lokasi memory pertama dari array untuk variable pointer.
- Dalam passing array, reference ke sebuah variable tidak menggunakan operator unary, karena dalam sebuah array memiliki alamat pertama pada lokasi memory dan nilai.

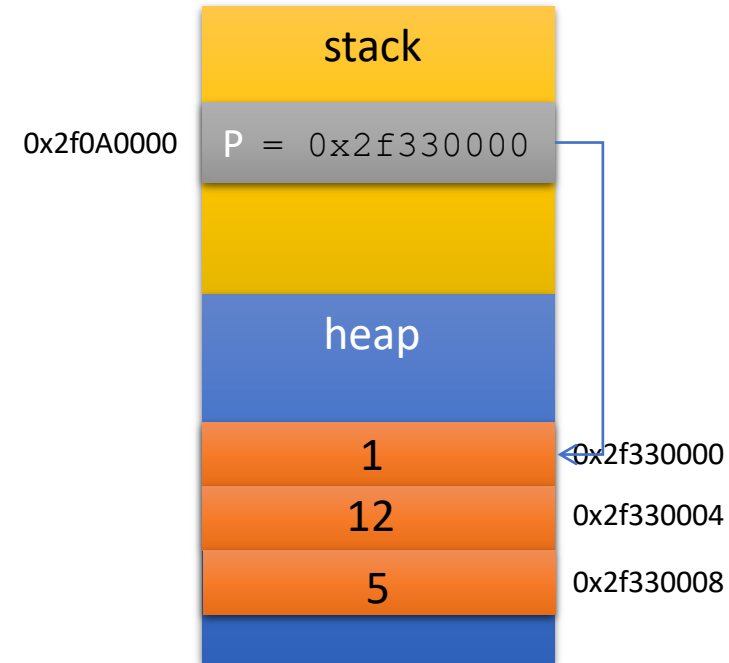
<pre>char name=0; scanf ("%c", &name);</pre>	<pre>char name[10]; scanf ("%s", name)</pre>
--	--

Pointer vs. Array

❑ Secara umum, pointer dapat digunakan seperti array

```
*p = *(p+0) = p[0]  
*(p + n) = p[n]
```

```
int main()  
{  
    int *p = new int [3];  
    p[0] = 1;  
    *(p + 1) = 12;  
    p[2] = 5  
}
```



Pointer vs. Array

```
int main()
{
    char a[3] = {1, 2, 3, 4};
    printf ("0x%x 0x%x %d\n", a, &a, *a);

    int *p = new int[3];
    p[0] = 1; p[1] = 2; p[2] = 3;
    printf ("0x%x 0x%x %d\n", p, &p, *p);

    int *p2 = (int*)a;
    printf("value of p2 = 0x%x\n", *p2);
}
```

Command prompt

```
0x14fd64 0x14fd64 1
0x591398 0x14fd60 1
Value of p2 = 0x04030201
```

- ❑ Array adalah sebuah pointer yang menunjuk ke dirinya sendiri
- ❑ Sebuah pointer dapat menunjuk ke sebuah alamat array

Pointer vs. Array

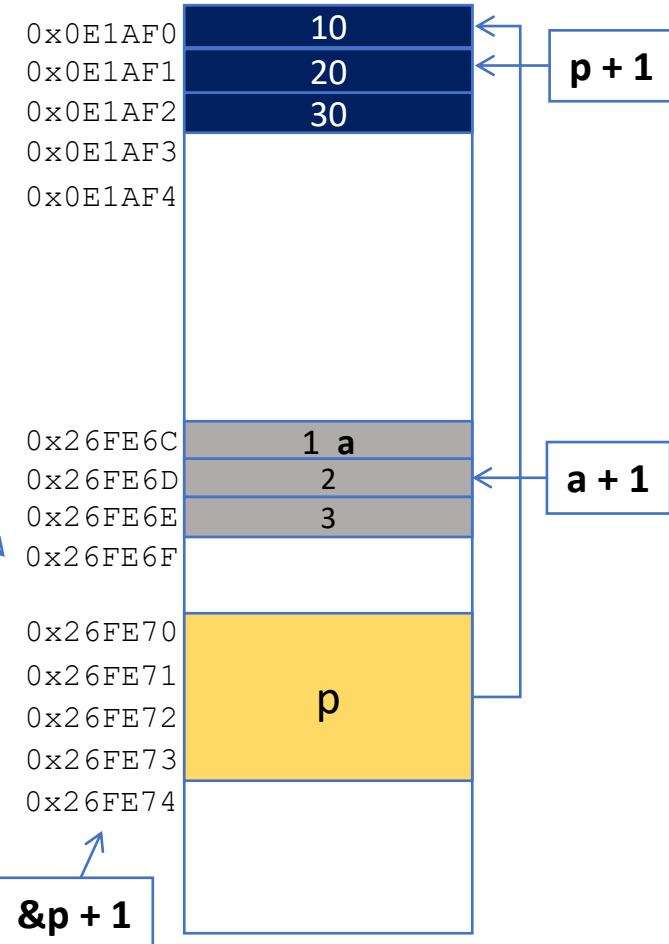
```
char a[3] = {1, 2, 3};

char *p = new char[3];
p[0] = 10; p[1] = 20; p[2] = 30;

printf ("a      = 0x%x      p = 0x%x\n", a, p);
printf ("a+1    = 0x%x      p+1 = 0x%x\n", a+1, p+1);
printf ("&a     = 0x%x      &p = 0x%x\n", &a, &p);
printf ("&a+1= 0x%x      &p+1 = 0x%x\n", &a+1, &p+1);
```

Command prompt

```
a      = 0x26FE6C      p = 0x0E1AF0
a+1    = 0x26FE6D      p+1 = 0x0E1AF1
&a     = 0x26FE6C      &p = 0x26FE70
&a+1= 0x26FE6F      &p+1 = 0x26FE74
```



Pointer vs. Array

- ❑ Karena keterbatasan dari stack, tidak dapat membuat array yang besar

```
int main()
{
    char arr[1034996];
}
```

FAIL

```
int main()
{
    char *p = new char[1034996];
}
```

OK

o Tidak dapat menghapus array

```
int main()
{
    char arr[100];
    delete arr;
}
```

FAIL

```
int main()
{
    char *p = new char[1034996];
    delete p;
}
```

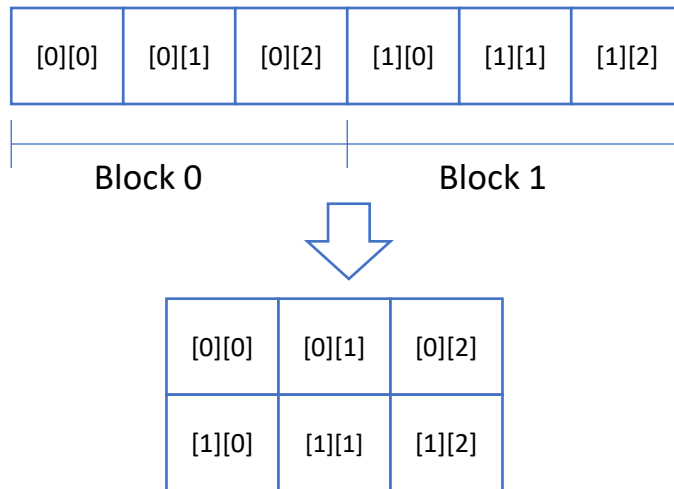
OK

- ❑ Blok memory dari array di bebaskan secara otomatis ketika sudah diluar skope out-of-scope
- ❑ Memory dinamis HARUS dibersihkan secara manual dengan memanggil “delete”

Pointer vs. Array

❑ 2D array

```
int arr[2][3]
```



2D array & 2D pointer dapat digunakan dengan cara yang sama

```
arr[2][2] = 5
```

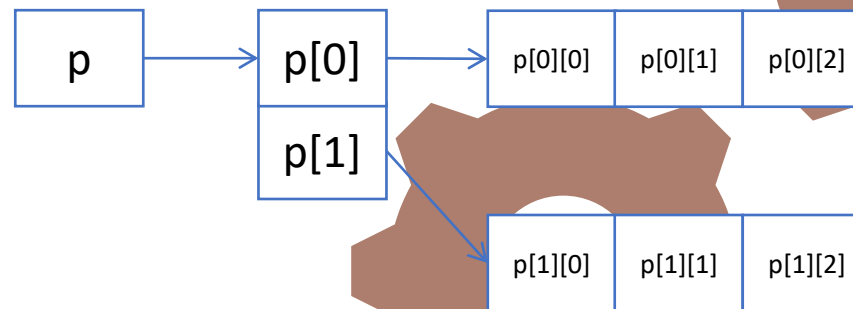
```
p[2][2] = 10
```

❑ pointer-to-pointer

```
int **p = new int*[2];
```

```
p[0] = new int[3];
```

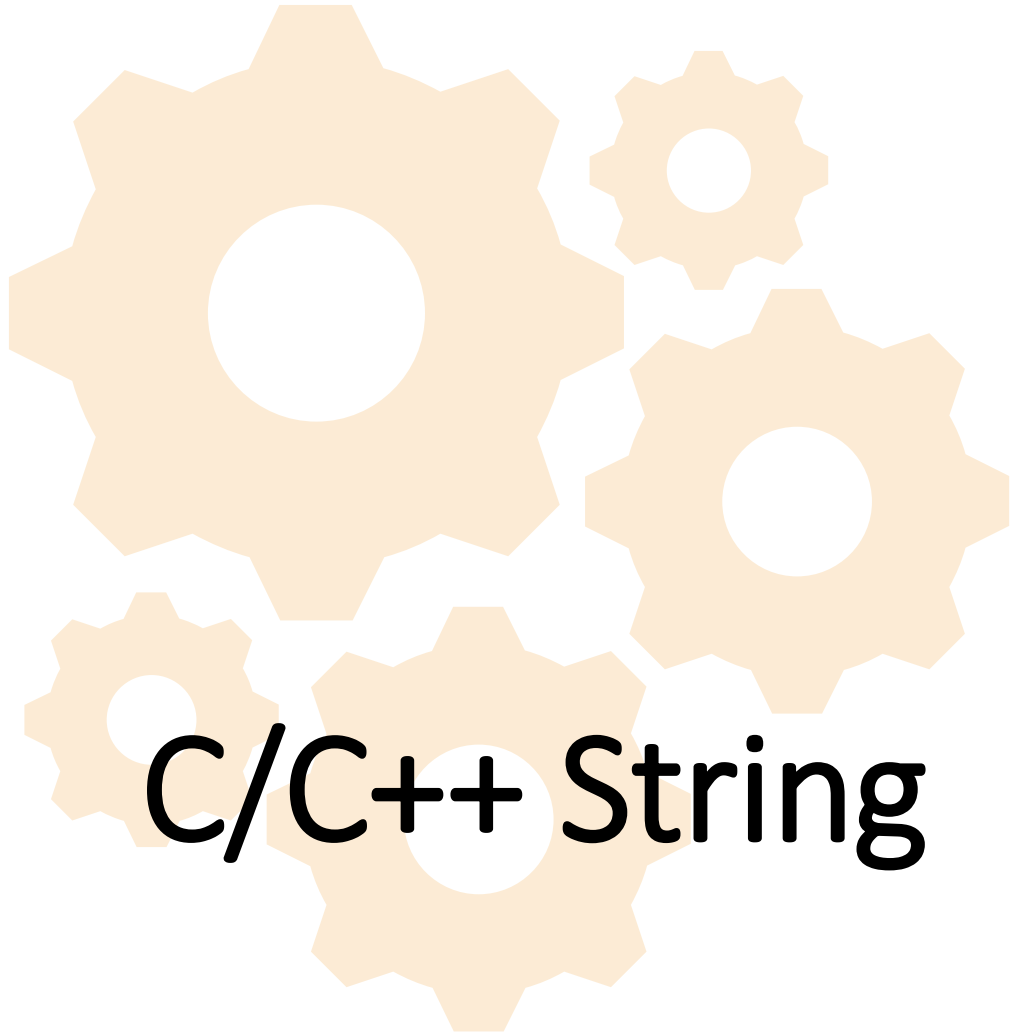
```
p[1] = new int[3];
```



$*(*(p + i) + j) \Leftrightarrow p[i][j]$

Latihan

- Buat Program Fungsi berdasarkan operasi berikut:
 - Deklarasi tiga pointer variable: **iPtr bertype int**, **cPtr bertype char**, **fPtr bertype float**
 - Deklarasi tiga variable baru: **iNumber bertype int**, **fNumber bertype float**, **cChar bertype char**
 - Menetapkan alamat dari masing – masing variable non-pointer ke variable pointer yang cocok
 - Print nilai dari tiap variable pointer dan non pointer
 - Print alamat dari tiap variable pointer dan non pointer
 - Terapkan **Passi By Value** dan **PassBy Reference** di dalam fungsi yang dibuat



C/C++ String

String

- ❑ Tidak ada standar string di C/C++
- ❑ Menggunakan `char*`, atau `char[]`
- ❑ String di C/C++ adalah array dari byte, diakhiri dengan `'\0'`

```
char *st = "String";
```



Alokasi String

❑ Alokasi Statis

```
char *st = "String";  
char st2[] = "String";
```

❑ Alokasi Dinamis

```
char *st3 = new char[6];  
st3[0] = 's';  
st3[1] = 't';  
st3[2] = 'i';  
st3[3] = 'n';  
st3[4] = 'g';  
st3[5] = '\\0';
```

Alokasi String (lanj)

```
char* GetString1()  
{  
    char *st = "String";  
    return st;  
}  
  
char* GetString2()  
{  
    char st[] = "String";  
    return st;  
}
```

```
char* GetString3()  
{  
    char *st = new char[6];  
    strcpy(st, "String");  
    return st;  
}
```

What are
different?

```
int main()  
{  
    printf("Say: %s", GetString1());  
    printf("Say: %s", GetString2());  
    printf("Say: %s", GetString3());  
}
```

Fungsi untuk utilitasi Memory

HARUS `#include <string.h>`

```
void * memcpy ( void * destination, const void * source, size_t num )
```

- ❑ Mengkopu nilai dari sejumlah bytes dari lokasi yang ditunjuk oleh sumber/source secara langsung ke blok memory yang ditunjuk oleh destinasinya/destination

```
int memcmp ( const void * ptr1, const void * ptr2, size_t num )
```

- ❑ Membandingkan String C yang ditunjuk oleh sumber/source didalam array yang ditunjuk oleh destinasinya/destination, termasuk karakter terminasi '\0'

Fungsi untuk utilitasi Memory

❑ `size_t strlen ( const char * str )`

- Mengembalikan panjang dari str
- Panjang dari String di C dihitung oleh mengakhiri null-character '\0'
- Seharusnya tidak perlu bingung dengan ukuran array yang menahan string-nya

❑ `char * strcpy ( char * destination, const char * source )`

- Mengkopi C string yang ditunjuk oleh sumber/source pada array yang ditunjuk oleh destinasinya/destination, termasuk mengakhir null character

❑ `int strcmp ( const char * str1, const char * str2 )`

- Membandingkan String C *str1* dengan String C *str2*.

<http://www.cplusplus.com/reference/cstring/>

Constant pointer vs. pointer to constant

❑ Constant pointer:

- Alamat dari memory disimpan secara konstan
- Nilai dari alamat yang “menunjukkan ke” dapat diubah

```
char char_A = 'A';  
char char_B = 'B';  
char * const myPtr = &char_A;  
myPtr = &char_B;    // error - can't change address of myPtr
```

❑ Pointer to constant:

- Nilai dari alamat yang “menunjukkan ke” adalah konstan
- Alamat dari memory disimpan tidak dapat dirubah

```
char char_A = 'A';  
const char * myPtr = &char_A;  
*myPtr = 'J'; // error - can't change value of *myPtr
```

Referensi

Utama :

1. Liem, Inggriani. Diktat Pemrograman Prosedural dan Fungsional Informatika Informatika ITB. IF-ITB. 2007
2. Bjarne Stroustrup, 2014, Programming: Principles and Practice Using C++ (Second Edition), Addison-Wesley Professional

Pendukung :

1. Introduction to Computer Science and Programming in Python, MIT <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-0001-introduction-to-computer-science-and-programming-in-python-fall-2016>
2. Introduction to Computer Science and Programming, MIT <https://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-00sc-introduction-to-computer-science-and-programming-spring-2011/index.htm>